



Experiment - 7

Student Name: Mohd Shahid

UID: 23BCS10258

Branch: BE-CSE

Section/Group: KRG-2A

Semester: 6th

Date of Performance: 26/03/2026

Subject Name: System Design

Subject Code: 23CSH-314

Aim: To design a scalable **ride-sharing platform** similar to **Uber/Ola** that enables real-time ride booking, driver matching, fare estimation, and live tracking with high availability and low latency.

Objectives:

- Understand the architecture of real-time ride-sharing systems.
- Identify functional requirements such as ride booking and driver matching.
- Identify non-functional requirements including scalability, latency, and availability.
- Analyze consistency trade-offs using CAP theorem.
- Design REST APIs for rider and driver workflows.

Procedure:

1. Analyzed the working and architecture of popular ride-sharing services like Uber and Ola.
2. Identified the core system components, including Rider, Driver, Ride, Location, and Fare.
3. Examined techniques for real-time location tracking using WebSocket communication.
4. Defined both functional and non-functional system requirements.
5. Designed API endpoints to handle ride booking, driver matching, and ride tracking.
6. Studied and compared different algorithms for matching riders with nearby drivers.
7. Evaluated consistency challenges involved in assigning rides within a distributed environment.

Functional Requirements:

1. Rider should be able to get fare estimation based on pickup and drop location.
2. Rider should be able to request a ride.
3. Rider should be able to choose ride categories (car, bike, auto, etc).
4. Driver should be able to accept or reject ride requests.
5. System should match riders with nearby available drivers.
6. System should support real-time tracking of rides.
7. Riders and drivers should be able to rate each other after trip completion.
8. System should support multiple payment methods.



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

Core Entities of the System

- User / Rider
- Driver
- Location (Pickup & Drop-off)
- Fare
- Ride / Trip

API Design

1. Rider API

A. Get Fare Estimation:

`GET /api/v1/fare?pickupLat={}&pickupLng={}&dropLat={}&dropLng={}`

B. Request Ride: `POST /api/v1/rides/request`

C. Ride History: `GET /api/v1/rides/history`

D. Cancel Ride: `POST /api/v1/rides/{ride_id}/cancel`

E. Rate Ride: `POST /api/v1/rides/{ride_id}/rating`

2. Driver API

A. Update Location (WebSocket): `WS /drivers/location`

B. Accept/Reject Ride: `POST /api/v1/rides/{request_id}/respond`

C. Start Ride: `POST /api/v1/rides/{ride_id}/start`

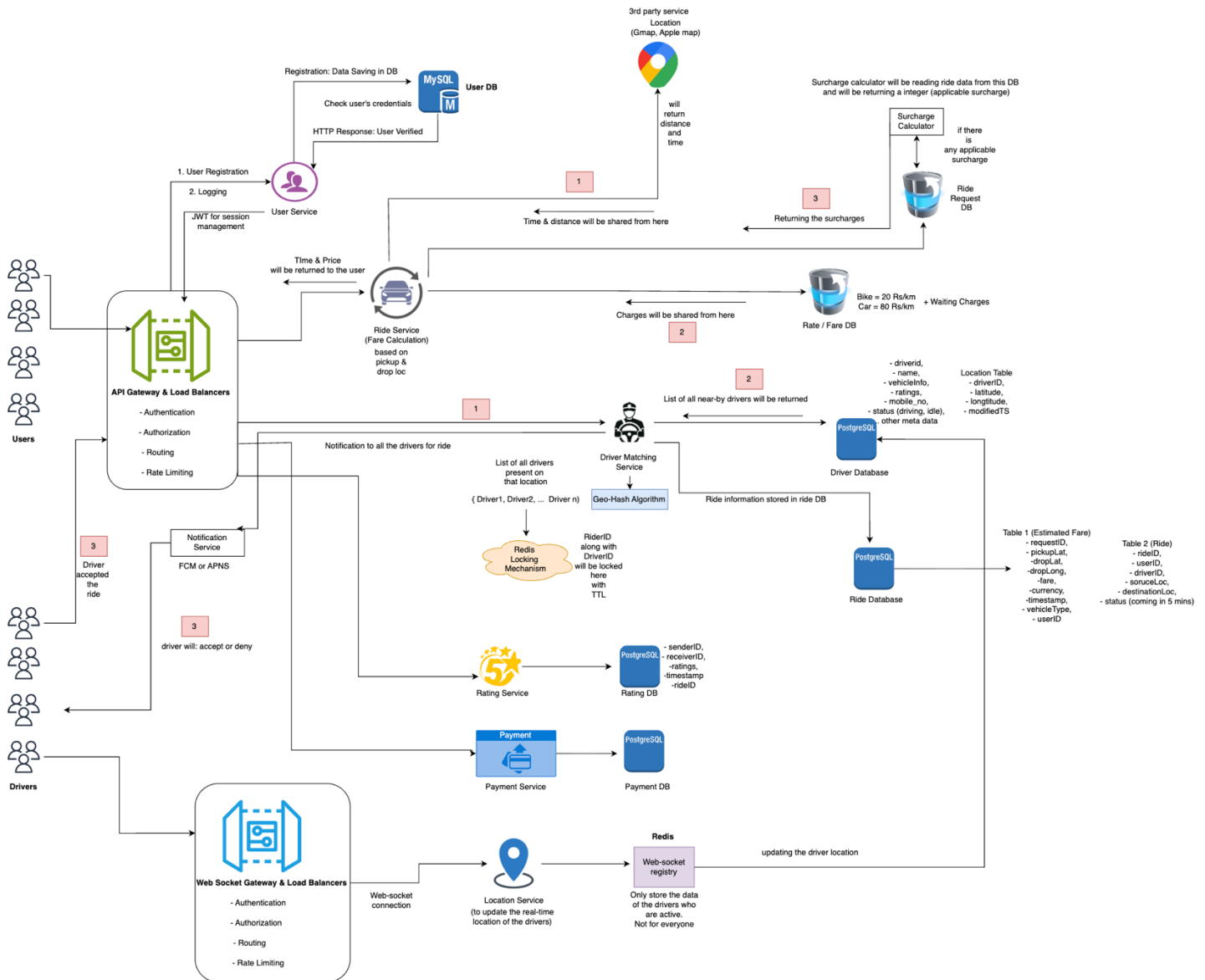
D. End Ride: `POST /api/v1/rides/{ride_id}/end`

Non-functional Requirements:

1. System should support up to 1.5 million users and drivers.
2. High availability for riders (eventual consistency).
3. Strong consistency for driver assignment (no double booking).
4. Driver matching latency should be less than 1 minute.
5. Real-time location updates should have minimal delay.

High Level Design (HLD):

The system consists of client applications (rider and driver apps), an API Gateway, and microservices such as User Service, Ride Service, Matching Service, Location Service, and Payment Service. Driver locations are updated via WebSockets and stored in a geo-indexed datastore. The Matching Service finds nearby drivers and assigns rides. Asynchronous processing using message queues ensures scalability, while payments and ratings are handled after trip completion.



Learning Outcomes:

- Designed a scalable ride-sharing system with real-time capabilities.
- Learned how ride booking and driver matching systems work.
- Understood trade-offs between consistency and availability.
- Implemented structured API design for rider and driver workflows.
- Learned about real-time systems using WebSockets and geo-indexing.